

Let's Frame the Framework

From an Idea to a Reusable iOS Framework

Bhavin Vaghela

Sr iOS App Developer & Assistant project manager @Hyperlink Infosystem

Bhavin Vaghela

Experience: 11 Years

**Sr. iOS Developer & Project
Manager**

At Hyperlink InfoSystem



Framework

What is a Framework?

A framework is a reusable collection of code, resources, and APIs that provides ready-made functionality for building applications.

Instead of building every feature from scratch, we can use frameworks to access existing functionality and focus on what makes their app unique.

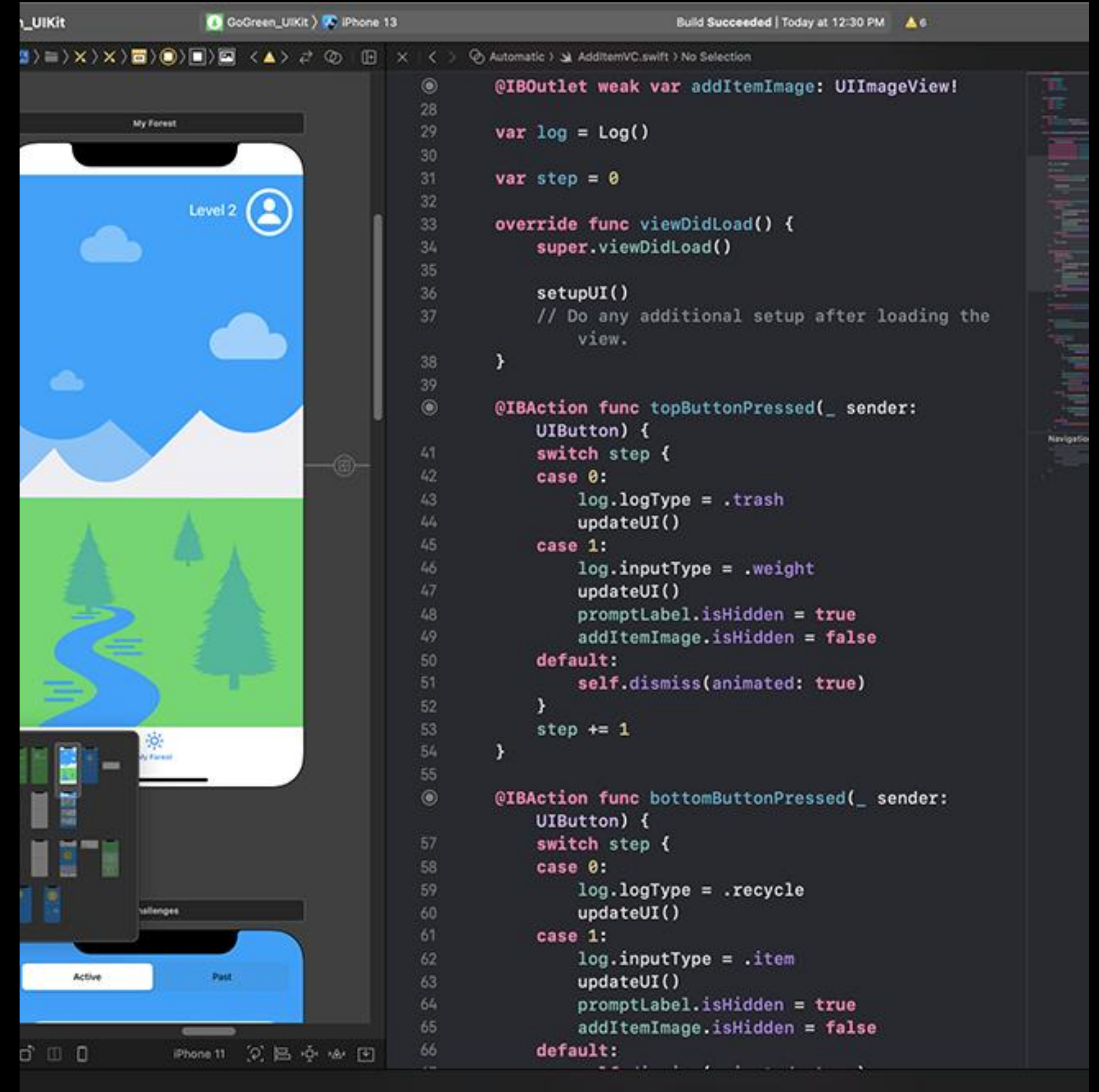
Key concepts

- **Pre-built functionality:** Ready-to-use classes, methods, and APIs for common development tasks.
- **Reusable and modular:** Use the same framework across multiple apps and projects.
- **Saves development time:** Avoid reinventing existing solutions and reduce duplicated code.

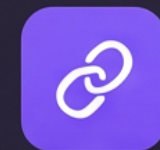
Examples

UI and app experience frameworks

- UIKit — UI components, view controllers, navigation, touch handling.
- SwiftUI — declarative UI, state-driven views and cross-platform interface development.
- AppKit — native macOS interfaces, menus and controls.

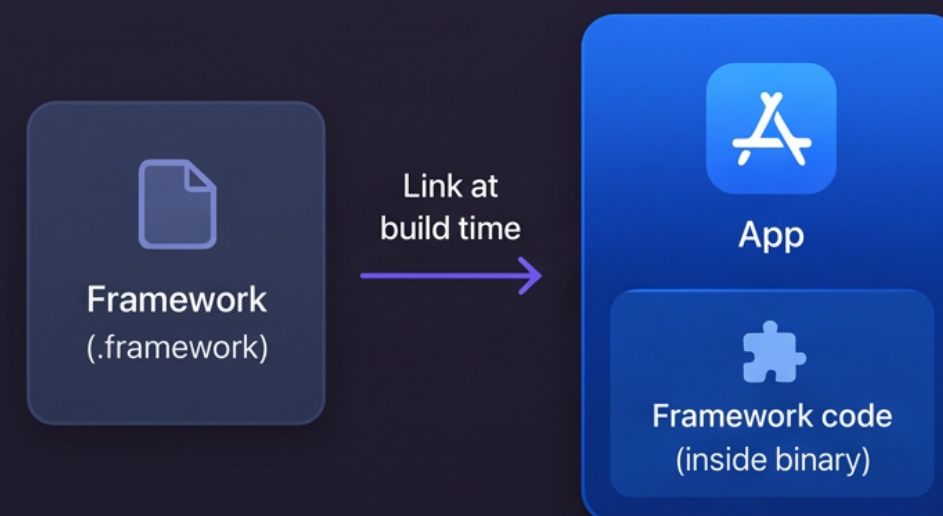


Types of Framework



Static Frameworks

Code is linked into the app during build time.



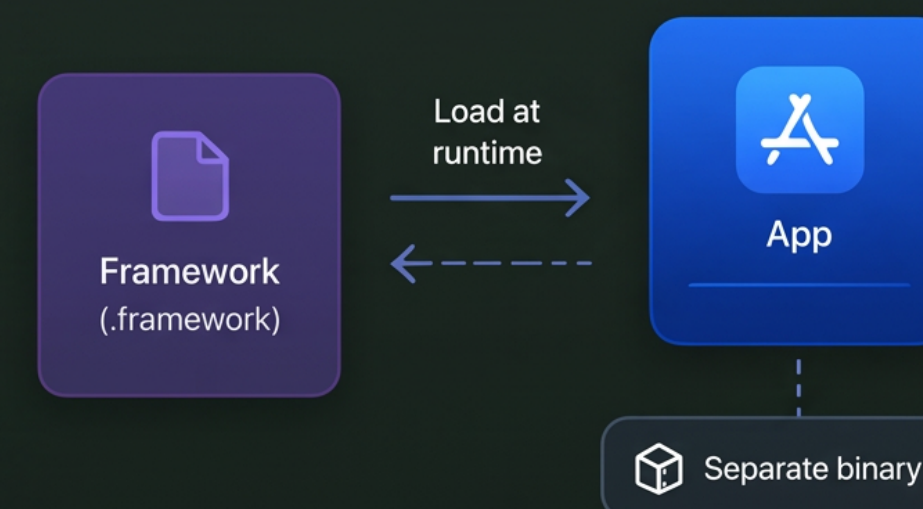
- ✓ Code becomes part of the app's executable.
- ✓ No separate framework file at runtime.
- ✓ Useful for smaller apps and simpler distribution.

Static Framework



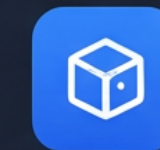
Dynamic Frameworks

Code is loaded separately at runtime by the operating system.



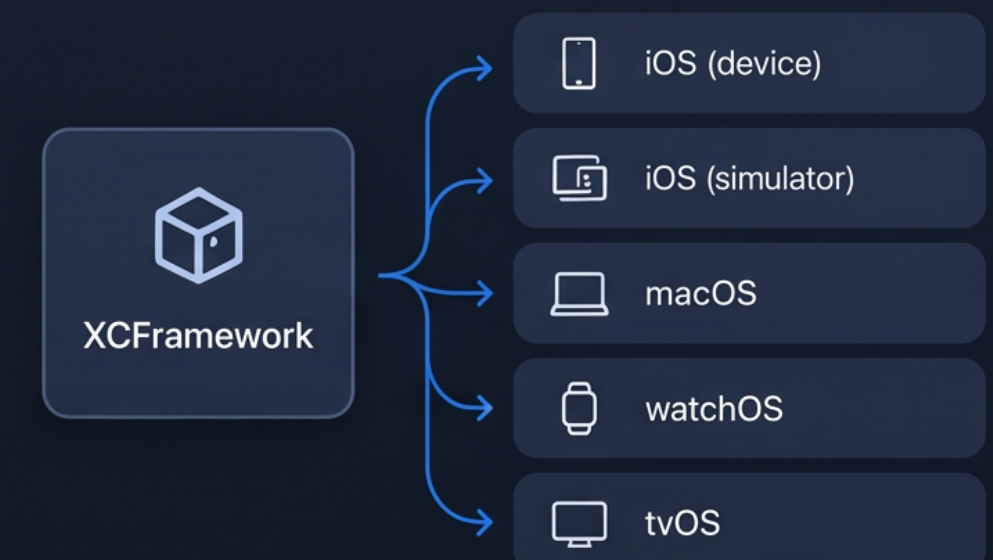
- ✓ Has a separate framework binary.
- ✓ Can be embedded and signed with the app.
- ✓ Allows modularity and easier updates (for some cases).

Dynamic Framework



XCFrameworks

A packaging format for distributing frameworks across multiple Apple platforms and architectures.



- ✓ Contains multiple platform-specific builds in a single bundle.
- ✓ Supports both static and dynamic libraries.
- ✓ Xcode selects the appropriate variant automatically.

XCFramework

Problems faced Old Framework

- Universal (Fat) Frameworks
- Platform-Specific Distribution
- Package Manager Binary Limitations

Universal (Fat) Frameworks

Traditional approach: lipo -create to combine device + simulator

- Single binary with multiple architectures (arm64 + x86_64)
- Bloated binary size (2x+ for unused arch)
- No support for Apple Silicon Macs (arm64 macOS)
- Cannot mix static + dynamic variants
- No platform-specific headers/resources
- Bitcode complications (deprecated but legacy issues)

Problems faced Old Framework

- Universal (Fat) Frameworks
- Platform-Specific Distribution
- Package Manager Binary Limitations

Platform-Specific Distribution

Before XCFramework, you need separate artifacts

- iOS Device: MyFramework.framework (arm64)
- iOS Simulator: MyFramework.framework (x86_64/arm64)
- macOS Intel: MyFramework.framework (x86_64)
- macOS Apple Silicon: MyFramework.framework (arm64)
- tvOS Device: MyFramework.framework (arm64)
- tvOS Simulator: MyFramework.framework (x86_64/arm64)

- 10+ separate .framework bundles to manage
- Manual distribution & versioning nightmare

Problems faced Old Framework

- Universal (Fat) Frameworks
- Platform-Specific Distribution
- Package Manager Binary Limitations

Package Manager Binary Limitations

- CocoaPods:
 - Binary pods require manual .framework zip hosting
 - No native multi-platform binary support
 - use_frameworks! forces dynamic linking
 - Simulator/device architecture conflicts
- Swift Package Manager (pre-XCFramework):
 - Only source distribution (no binary targets)
 - Cannot distribute closed-source frameworks
 - Build from source = slow CI, IP exposure

Solution?

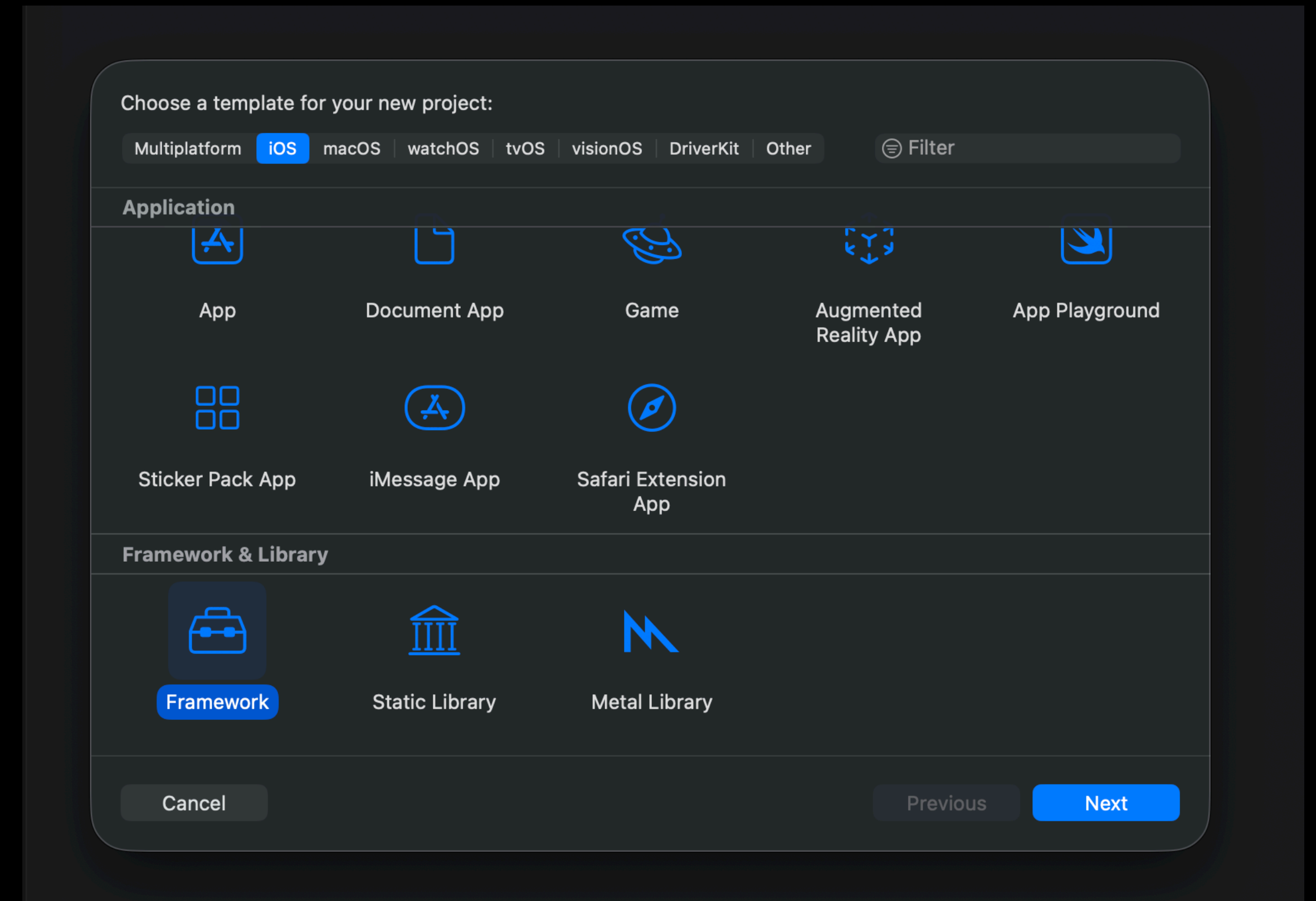
XCFramework: The Unified Solution

XCFramework makes binary distribution incredibly easy because it shifts all the heavy lifting from the developer onto Xcode itself. It acts as a smart container that automatically manages platforms and architectures.

- Single .xcframework package for ALL platforms
- Native Xcode & SwiftPM support
- App Store validated format
- Supports static + dynamic in one package
- Platform-aware (not just architecture-aware)

How to Create a framework

- Open Xcode.
- Select File → New → Project.
- Select the Framework template under the appropriate platform section, such as iOS.
- Click Next.
- Enter the project details
- Click Create.



What to Take Care Of

Public API & Interface Design

- Expose only the types and methods clients need
- Use access control intentionally: public vs. internal
- Keep the public API small, clear, and well documented
- Avoid breaking changes once clients depend on the API
- Enable library evolution support when distributing a binary

What to Take Care Of

Platform, Architecture & Build Setup

- Define supported Apple platforms and deployment targets
- Build device and simulator variants separately
- Include the correct architectures for each destination
- Use an archive build for each platform variant
- Combine archives with `-create-xcframework` — never lipo device and simulator binaries together

What to Take Care Of

Dependencies, Resources & Linking

- List every third-party and Apple framework dependency
- Confirm each dependency supports every target platform
- Bundle assets, localization and required resources correctly
- Load resources reliably from the framework bundle at runtime
- Choose static or dynamic linkage; verify embedding, signing and duplicate symbols

What to Take Care Of

Validation, Distribution & Maintenance

- Test the XCFramework in a separate sample app
- Verify platform slices, architectures and minimum OS versions
- Check supported Xcode and Swift toolchain compatibility
- Package, sign and publish a clean release artifact
- Document installation, versioning and breaking changes

Idea to Framework



```
17 public struct AboutView: View {
25     public var body: some View {
71         Section {
72            ForEach(aboutPage.members, id: \.self) { member in
73                 HStack(alignment: .firstTextBaseline) {
74                     Image(systemName: "person.text.rectangle.fill")
75                         .imageScale(.large)
76                         .foregroundColor(.secondary)
77
78                     VStack(alignment: .leading) {
79                         Text(member.name)
80                         Text(member.about)
81                             .font(.caption)
82                             .foregroundColor(.secondary)
83                     }
84                     Spacer()
85                     Button {
86                         store.send(.user(.socialLinkTapped(
87                             member.socials.linkedin)))
88                     } label: {
89                         Image(systemName: "paperplane.fill")
90                     }
91                     .shineDecoration()
92                 }
93             } header: {
```

```
0
1 class LoginVC: UIViewController {
2
3     //MARK: Outlet
4     @IBOutlet weak var btnLogin: UIButton!
5
6
7     //MARK: LifeCycle Methods
8     override func viewDidLoad() {
9         btnLogin
10         .backgroundColor(color: .black)
11         .setTitle("Login", for: .normal)
12     }
13
14 }
15
```

Live Demo



Pre-Distribution Checklist

What to validate before shipping

- Platforms and targets
- Release build artifacts
- dSYM strategy
- Code signing needs
- Semantic versioning
- Device and sim tests



<https://www.linkedin.com/in/bhavin-vaghela/>

Thank you